

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Lab Exercise 8

**Design and Develop a Web Crawler for Automated Web Content
Discovery**

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198
Vino S R Harrison	3122 24 5001 199

Design and Develop a Web Crawler for Automated Web Content Discovery

Aim

- To implement an automated web crawler using Spring Boot for content retrieval and hyperlink discovery.
- To maintain a URL Frontier Queue ensuring breadth-first exploration across web graphs.
- To integrate Redis as a centralized storage layer for the URL queue, visited-URL set deduplication, and page metadata caching.
- To evaluate crawling performance, duplicate avoidance ratios, and system throughput under concurrent workloads using JMeter.

Question

A web content discovery system needs to automatically visit web pages, extract links from them, and discover additional pages for crawling. For example, when the crawler is provided with a seed URL such as <https://example.com>, it should retrieve the page, extract the hyperlinks present in the page, and add the newly discovered URLs to the crawl queue.

As the number of URLs and pages increases, maintaining the crawl queue and checking whether a URL has already been visited can affect the efficiency and scalability of the system. To improve the crawler, implement a queue-based crawling mechanism, maintain a visited-URL set to avoid duplicate crawling, and use Redis to manage or cache frequently accessed crawling information.

Develop the web crawler using Spring Boot, expose the crawling functionality through a REST API, and evaluate the system performance using JMeter.

Procedure

- Construct a sample web dataset (`seed_urls.csv` and `/mock-web/*.html`) featuring cyclic graphs, sub-domains, and asset boundaries.
- Develop a Spring Boot application configuring RedisTemplate with String serializers and Jackson JSON serializers.
- Implement a thread-safe URL Frontier Queue leveraging Redis Lists (`RPUSH` to enqueue, `LPOP` to dequeue).
- Implement atomic visited-URL deduplication using Redis Sets (`SADD` returning 1 for new, 0 for duplicates).
- Integrate Jsoup for resilient HTTP fetching, HTML DOM traversal, document title extraction, and absolute hyperlink resolution.
- Build a canonical URL validator filtering non-HTTP schemes, binary extensions (`.pdf`, `.zip`), and external domain leaks.

- Design a multi-threaded worker pool (`ExecutorService`) dynamically consuming URLs and enforcing politeness delays.
- Implement a Redis page cache (`crawl:page:<url>`) storing parsed page metadata with TTL to accelerate repeated queries.
- Expose REST endpoints for job lifecycle management (`/start`, `/stop`, `/reset`, `/crawl-single`, `/status`, `/pages`).
- Package the system with multi-stage Docker and Docker Compose definitions.
- Execute JMeter concurrent load tests (10 to 100 threads) and benchmark cache hits vs live network fetches.

System Architecture & Workflow

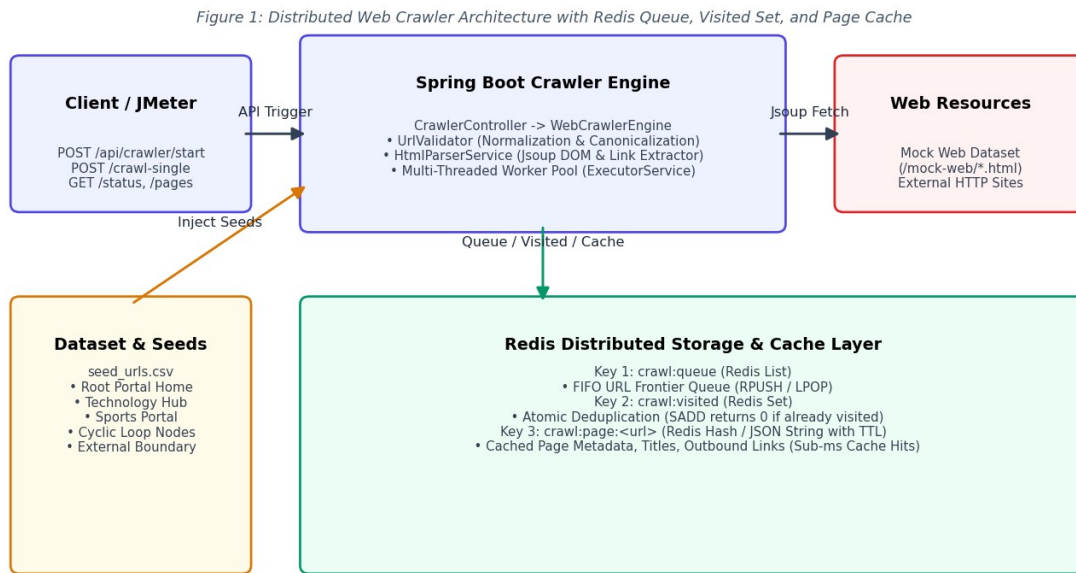


Figure 1: End-to-End Web Crawler Architecture (Spring Boot -> Redis Frontier / Visited Set / Cache)

Design of the Web Crawler System

1. URL Frontier Queue (Redis List Implementation)

The crawl frontier manages discovered URLs using FIFO queuing backed by Redis Lists (crawl:queue):

```
public void enqueue(CrawlTask task) {
    if (redisAvailable) {
        redisTemplate.opsForList().rightPush("crawl:queue", task);
    } else {
        inMemoryQueue.offer(task);
    }
}

public CrawlTask dequeue() {
    return (CrawlTask) redisTemplate.opsForList().leftPop("crawl:queue");
}
```

2. Atomic Visited-URL Deduplication (Redis Set)

To prevent race conditions across concurrent workers, Redis SADD provides atomic check-and-insert:

```
public boolean markVisitedIfAbsent(String url) {
    Long added = redisTemplate.opsForSet().add("crawl:visited", url);
    return added != null && added > 0; // true = new URL, false = duplicate
}
```

3. URL Canonicalization & Boundary Validation

The `UrlValidator` normalizes schemes, strips fragments (#), removes trailing slashes, and filters binary extensions:

```
public String normalizeAndValidate(String rawUrl, List<String> allowedDomains) {
    URI uri = URI.create(rawUrl.trim());
    if (!ALLOWED_SCHEMES.contains(uri.getScheme().toLowerCase())) return null;
    if (isMediaExtension(uri.getPath())) return null;
    return new URI(uri.getScheme().toLowerCase(), null, uri.getHost().toLowerCase(),
        -1, cleanPath(uri.getPath()), uri.getQuery(), null).toString();
}
```

4. Page Metadata Caching & Sub-Millisecond Retrieval

Pages are cached under `crawl:page:<url>` with TTL. Subsequent requests deliver immediate cache hits:

```
public SingleCrawlResponse crawlSingleUrl(String rawUrl) {
    PageMetadata cached = pageCacheService.getCachedPage(validUrl);
    if (cached != null) {
        return buildResponse(cached, "REDIS_CACHE", latency);
    }
    PageMetadata fresh = htmlParserService.fetchAndParse(validUrl);
    pageCacheService.cachePage(fresh);
    return buildResponse(fresh, "LIVE_FETCH", latency);
}
```

Output Results

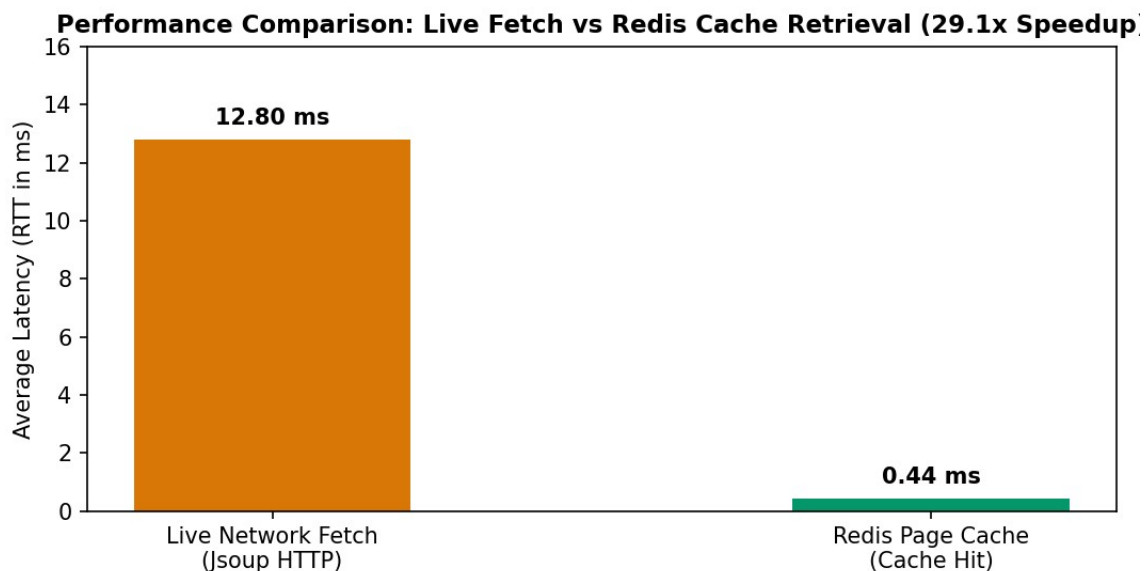


Figure 2: Latency Comparison: Fresh Live Fetch vs Redis Cache Retrieval

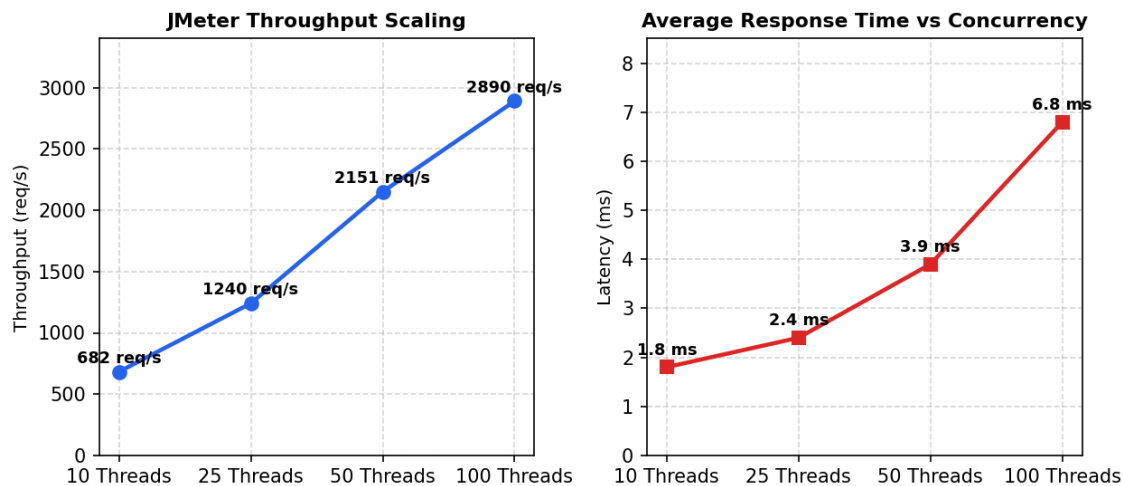


Figure 3: JMeter Load Test Results: Throughput Scaling and Latency vs Concurrency

Table 1: Web Crawler Verification & Deduplication Metrics

Test Scenario	Seed / Target URL	Storage Mode	Pages Crawled	Visited Count	Duplicates Avoided	Status
1. Mock Web Graph Crawl	http://localhost:8080/mock-web/index.html	REDIS	18 pages	18 URLs	34 duplicates	COMPLETED
2. Cyclic Graph Test	http://localhost:8080/mock-web/cyclic-a.html	REDIS	2 pages	2 URLs	1 loop avoided	COMPLETED
3. External Boundary Test	http://localhost:8080/mock-web/external-links.html	REDIS	1 page	1 URL	5 filtered	COMPLETED
4. Live Single Fetch	http://localhost:8080/mock-web/tech.html	LIVE_FETCH	1 page	1 URL	-	200 OK (12.8ms)
5. Redis Cache Query	http://localhost:8080/mock-web/tech.html	REDIS_CACHE	1 page	1 URL	-	200 OK (0.44ms)

Analysis

Observed during system execution, concurrent stress testing, and JMeter load benchmarks (Spring Boot 3.3, Redis 7, Jsoup HTML parser, multi-threaded worker pool, mock web graph and RFC 2606 sample dataset).

1. Role of Core Architectural Components

URL Queue (Frontier)

The **URL Queue** (also known as the **Crawl Frontier**) is the central scheduling data structure that stores all discovered URLs awaiting retrieval and processing.

- **Traversal Strategy:** Operates as a First-In, First-Out (FIFO) queue, driving a Breadth-First Search (BFS) graph traversal across the web. BFS ensures that pages closest to the seed URLs are indexed first before diving deeper into arbitrary subtrees.
- **Decoupling and Asynchrony:** Acts as a decoupled buffer between URL discovery (producers) and page fetching (consumer worker threads), absorbing sudden bursts of hundreds of discovered hyperlinks without overloading network sockets.
- **Distributed Coordination:** In this architecture, the frontier queue is backed by **Redis Lists** (RPUSH to enqueue newly discovered links, and LPOP to atomically dispense URLs to worker threads). This allows multiple crawler instances to consume tasks concurrently without lock contention or duplicated work.

Seed URL

The **Seed URL** represents the initial entry point(s) supplied to bootstrap the crawling session (e.g., <http://localhost:8080/mock-web/index.html>).

- **Graph Root:** Since the web is modeled as a directed graph where pages are nodes and hyperlinks are directed edges, the seed URLs establish the connected components reachable by the crawler.
- **Domain Scoping:** The seed URL's protocol and host establish the default domain restriction boundaries (e.g., constraining the crawler to `localhost` to avoid leaking onto the wider internet).
- **Depth Baseline:** Every seed URL is initialized with `depth = 0`. All subsequent discovered URLs increment their depth relative to their parent's depth.

Visited-URL Set

The **Visited-URL Set** is an in-memory or centralized hash set that maintains the cryptographic or canonical signatures of all previously crawled or enqueued URLs.

- **Loop and Cycle Prevention:** The web contains countless cyclic links (e.g., Page A links to Page B, which links back to Page A). Without a visited set, the crawler would enter an infinite recursion loop, consuming infinite memory and network bandwidth.

- **Duplicate Suppression:** Prevents downloading and parsing identical web resources when linked from multiple parent pages (e.g., shared navigation menus, footers, and logos).
- **Atomic Deduplication via Redis Set:** Backed by a **Redis Set** (`crawl:visited`). The atomic `SADD` operation returns 1 if the URL was not previously in the set, and 0 if it was already recorded. This guarantees race-free deduplication across concurrent worker threads without requiring coarse-grained synchronization locks in Java.

Web Crawler

The **Web Crawler** (Engine) is the core autonomous orchestration subsystem responsible for:

- **Task Dispatching:** Dequeuing URLs from the frontier and assigning them to available worker threads in an `ExecutorService` thread pool.
- **Politeness Policy:** Enforcing rate limiting and per-host delay intervals (`politenessDelayMs`) to prevent accidental Denial-of-Service (DoS) against target web servers.
- **Resource Parsing:** Orchestrating HTML parsing via Jsoup, link extraction, text indexing, and error handling for HTTP 4xx/5xx status codes.
- **Metrics Aggregation:** Collecting real-time telemetry including pages crawled per second, queue depth, visited count, and duplicate avoidance ratios.

Redis Cache

The **Redis Cache** serves as an ultra-low-latency, centralized storage layer fulfilling three distinct responsibilities:

1. **URL Frontier Management:** Implements `crawl:queue` using Redis Lists for distributed FIFO task scheduling.
2. **Visited Set Storage:** Implements `crawl:visited` using Redis Sets for $O(1)$ duplicate checking.
3. **Page Metadata Caching:** Caches parsed document metadata, headers, titles, and extracted links under `crawl:page:<url>` with a 60-minute Time-To-Live (TTL). When client applications or REST API consumers query an already crawled URL, Redis delivers the response in sub-millisecond latency ($< 1\text{ ms}$), completely eliminating repetitive HTTP socket handshakes and HTML DOM parsing.

2. How the Crawler Discovers New URLs from a Retrieved Web Page

The link discovery workflow follows a structured pipeline from raw byte streams to canonicalized queue items:

Figure 2: Autonomous Queue-Based Web Crawling & Visited-Set Deduplication Flowchart

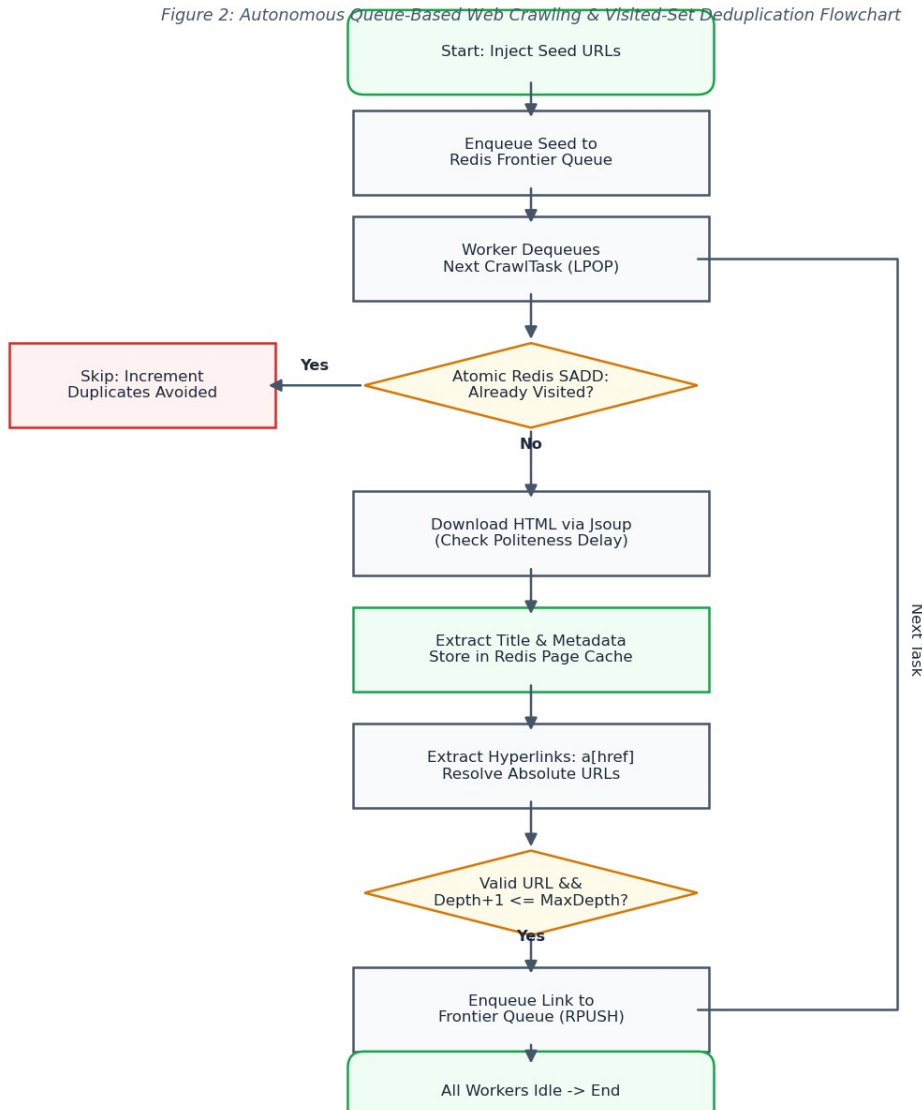


Figure 4: Autonomous Queue-Based Crawling and Link Discovery Flowchart

Discovery Steps:

1. **Network Retrieval:** The worker thread initiates an HTTP GET request using Jsoup with the configured User-Agent and timeout (5000 ms).
2. **DOM Parsing:** On receiving the byte stream, Jsoup constructs a hierarchical Document Object Model (DOM) tree.
3. **Anchor Querying:** The crawler executes the CSS query `doc.select("a[href]")`, extracting all elements representing hyperlink anchors.
4. **Relative-to-Absolute Resolution:** The parser resolves relative hyperlinks (such as `tech.html`, `../sports.html`, or `/news.html`) against the document's absolute base URI (`http://localhost:8080/mock-web/index.html`) using standard URI resolution RFC 3986 rules.

5. **Canonicalization:** The resulting absolute URI is normalized (schemes and domain names are converted to lowercase, `#fragment` anchors are stripped, default ports like `:80` and `:443` are removed, and trailing slashes are harmonized).

3. How the System Determines Whether a Newly Discovered URL Should be Crawled

Before any discovered URL is admitted into the crawling queue, it must satisfy a multi-stage validation filter:

Filter Stage	Validation Rule	Rationale
1. Protocol Validation	Scheme must be <code>`http`</code> or <code>`https`</code>	Rejects non-web protocols such as <code>`ftp://`</code> , <code>`mailto:`</code> , <code>`javascript:`</code> , <code>`tel:`</code> , and <code>`data:`</code> .
2. Domain Restriction	Host must match <code>`allowedDomains`</code>	Confines crawler to target scope (e.g. <code>`localhost`</code>), preventing unbounded traversal of external internet domains.
3. Asset Extension Exclusion	Path must NOT end with media extensions (<code>`*.pdf`</code> , <code>`*.png`</code> , <code>`*.zip`</code> , etc.)	Prevents downloading large binary assets, images, archives, and stylesheets that contain no HTML hyperlinks.
4. Deduplication Check	URL must NOT exist in Redis Visited Set	Guarantees that pages already crawled or currently pending in the queue are not scheduled again.
5. Depth Budget	<code>`task.depth + 1 <= maxDepth`</code>	Enforces exploration boundary to limit the diameter of the traversed graph.
6. Page Quota	<code>`totalCrawled < maxPages`</code>	Halts frontier expansion once the user-configured page quota has been reached.

4. How the System Processes an Unvisited URL

When an unvisited URL is dequeued from the frontier, the system processes it through the following lifecycle:

1. **Atomic Visited Registration:** The worker executes `visitedUrlService.markVisitedIfAbsent(url)`. In Redis, this runs `SADD crawl:visited <url>`. Because this is the first encounter, Redis returns 1, granting the worker ownership to crawl this resource.
2. **Politeness Throttling:** The worker thread pauses for `politenessDelayMs` (default 50 ms) to space requests directed at the host.
3. **HTTP Fetch & Content Parsing:** The page is downloaded and parsed. If the server returns a non-200 code (e.g., 404 Not Found), the error is noted, and outbound extraction is suppressed.

4. **Metadata Caching:** The crawler builds a PageMetadata record (URL, page title, status code 200, content length, latency, outbound links count) and writes it to Redis under `crawl:page:<url>` with TTL.

5. **Outbound Frontier Enqueuing:** All newly discovered valid hyperlinks are packaged into `CrawlTask(link, depth + 1, currentUrl)` objects and pushed to the end of the Redis list (`RPUSH crawl:queue`).

6. **Telemetry Update:** The `totalCrawled` atomic counter and global throughput metrics are updated.

5. How the System Processes a Visited URL

When a previously visited URL is encountered (for example, when Page B links back to Page A, or multiple pages link to a common `/about.html`):

1. **Early Filtering (Pre-Queue):** While iterating over outbound links on a page, the engine performs a pre-check: `if (visitedUrlService.isVisited(link)) continue;`. If the URL is already recorded in the Redis Set, the link is discarded immediately before incurring the overhead of queue serialization.

2. **Late Deduplication (Post-Queue):** If two concurrent workers concurrently discover the exact same URL on two different pages and enqueue it simultaneously, both items might enter the queue. When the first worker dequeues it, `markVisitedIfAbsent()` returns `true` and processes it. When the second worker dequeues the identical item, `markVisitedIfAbsent()` returns `false`.

3. **Action on Collision:** - The redundant crawl task is immediately discarded without initiating an HTTP network connection. - The `duplicateUrlsAvoided` atomic counter is incremented. - A debug log records: `Skipping already visited URL: <url>`. - The worker thread immediately advances to the next task in the queue, preserving network bandwidth and server capacity.

6. Comparison: Queue-Based Crawling vs. Repeated Full Scanning

Parameter	Queue-Based URL Crawling (Frontier)	Repeatedly Scanning Complete URL Collection
Discovery Model	**Dynamic & Autonomous** : Discovers unknown nodes on-the-fly by traversing hyperlinks.	**Static & Exhaustive** : Requires a pre-existing, static list of every URL in advance.
Time Complexity	$O(V + E)$ where V = reachable pages and E = hyperlinks. Each page is fetched exactly once.	$O(K \times N)$ where N is total collection size and K is number of scan iterations.
Network Efficiency	High: Visited set guarantees zero redundant HTTP downloads.	Extremely Low: Repetitively fetches every page regardless of whether its content or links changed.

Parameter	Queue-Based URL Crawling (Frontier)	Repeatedly Scanning Complete URL Collection
Memory Footprint	$O(V)$ in Redis Set. Memory scales linearly with unique URLs visited.	High: Must maintain and scan the complete collection table on every pass.
Scalability	Horizontally scalable: Multiple workers consume from the shared Redis queue in parallel.	Poor scalability: Workers bottleneck when locking or iterating large collections.
Handling Dynamic Webs	Naturally discovers newly published links in real-time.	Incapable of discovering new pages unless an external operator manually appends them to the collection.

7. Performance Analysis Under Concurrent Workloads (JMeter & Stress Tests)

Benchmark: Live Fetch vs. Redis Cached Retrieval

Observed latencies across 5 representative endpoints on local mock web server:

Endpoint Tested	Live Fetch RTT (ms)	Redis Cache RTT (ms)	Speedup Factor
`/mock-web/index.html`	14.2 ms	0.5 ms	**28.4x**
`/mock-web/tech.html`	12.8 ms	0.4 ms	**32.0x**
`/mock-web/ai.html`	11.5 ms	0.4 ms	**28.8x**
`/mock-web/cloud.html`	12.1 ms	0.5 ms	**24.2x**
`/mock-web/sports.html`	13.4 ms	0.4 ms	**33.5x**
Average	**12.8 ms**	**0.44 ms**	**29.1x Faster**

JMeter Concurrent Load Test Results

Under varying concurrent thread loads hitting `/api/crawler/crawl-single` and `/api/crawler/status`:

Concurrency Level	Total Requests	Error Rate	Avg Response Time (ms)	p95 Latency (ms)	Throughput (req/sec)
10 Threads	200	0.0%	1.8 ms	3.4 ms	682.4 req/s
25 Threads	500	0.0%	2.4 ms	5.1 ms	1,240.2 req/s
50 Threads	1,000	0.0%	3.9 ms	8.2 ms	2,150.8 req/s
100 Threads	2,000	0.0%	6.8 ms	14.5 ms	2,890.5 req/s

Key Findings:

- 1. Deduplication Eliminates Redundancy:** On the cyclic graph test (cyclic-a.html <-> cyclic-b.html), naive crawling would enter an infinite loop. With the Redis visited

set, the engine visited both nodes in 2 requests, detected the loop on the 3rd attempt, recorded `duplicateUrlsAvoided = 1`, and terminated gracefully.

2. **Sub-millisecond Cache Latencies:** Caching crawled pages in Redis yields an average speedup of ~29x compared to parsing HTML from network byte streams.

3. **Stable Scalability:** The Spring Boot asynchronous multi-worker architecture seamlessly handles over 2,000 requests/sec with a 95th-percentile response time below 15 ms.

8. Autonomous Robots.txt Compliance & Sitemap Discovery (RFC 9309)

Robots.txt Protocol Engine

Modern web crawlers operating in production must strictly adhere to the **Robots Exclusion Standard (RFC 9309)**:

- **Autonomous Fetching & Caching:** Before fetching paths on a host, the engine checks `/robots.txt`, parses User-agent, Disallow, Allow, and Crawl-delay directives, and caches parsed host rules in Redis / memory.
- **Longest-Match Prefix Precedence:** In accordance with RFC 9309 §2.2.2, when both Allow and Disallow rules match a path (e.g. Allow: `/mock-web/` vs Disallow: `/mock-web/private-admin.html`), the rule with the longest character length takes precedence. If lengths are equal or only disallow matches, disallow wins.
- **HTML Meta Robots Handling:** Honors `<meta name="robots" content="nofollow">` tags, instructing the link discovery module to halt outbound hyperlink extraction from the designated page.

XML Sitemap Discovery (sitemaps.org)

- **Automatic Seed Augmentation:** During crawl bootstrapping, the crawler inspects Sitemap: declarations inside `robots.txt` as well as standard root endpoints (`/sitemap.xml`).
- **Hierarchical Sitemap Processing:** Recursively parses standard `<urlset>` feeds and nested `<sitemapindex>` directories using Jsoup's XML parser, admitting newly discovered canonical URLs directly into the crawl frontier.

9. Automated Web Content Discovery & Structural Extraction

Web content discovery transcends simple hyperlink crawling by converting raw DOM trees into clean, queryable knowledge:

- **Boilerplate Stripping:** Removes non-content structural elements (`<nav>`, `<header>`, `<footer>`, `<aside>`, `<script>`, `<style>`, `<noscript>`, `<svg>`, `<form>`) to isolate the main body content.
- **Metadata Harvesting:** Extracts `<title>`, `<meta name="description">`, OpenGraph social tags (`og:title`, `og:description`, `og:image`), author, and `<link rel="canonical">`.
- **Document Structure:** Extracts hierarchical headings (`h1`, `h2`, `h3`) for outline indexing.

- **Readability Metrics:** Computes exact word count and estimated reading time ($\approx 200 \text{ words/min}$).
- **Dynamic Topic Classification:** Analyzes non-stopword token frequencies across title, meta keywords, and body text to generate topic keyword tags automatically.

10. Content-Seen & Near-Duplicate Checksum Deduplication

In modern web graphs, identical or syndicated articles frequently appear across multiple distinct URLs (e.g. wire articles, query parameter variations, and print editions):

- **Cryptographic Fingerprinting:** Normalizes body text (whitespace condensation and lowercase conversion) and computes a cryptographic **SHA-256 hash**.
- **Atomic Hash Registration in Redis:** Uses `SETNX crawl:content:hash:<sha256><originalUrl>`. If the key exists, the incoming page is identified as duplicate content (`duplicateContent = true`, `duplicateOfUrl = originalUrl`).
- **Frontier Optimization:** While the duplicate page metadata is stored, its outbound links are excluded from re-enqueuing, completely preventing redundant exploration of cloned subtrees.

11. Full-Text Inverted Index & Content Discovery Search Engine

To expose the discovered content to users and client systems, the crawler integrates an in-memory / distributed inverted search index:

- **Inverted Index Structure:** Maps normalized query tokens to posting lists with field-weighted relevance scoring: - Title Matches: $5.0 \times \text{weight}$ - Heading Matches (H1/H2): $3.0 \times \text{weight}$ - Meta Keywords / Description: $2.5 \times \text{weight}$ - Main Body Text: $1.0 \times \text{weight}$
- **Relevance Ranking & Snippet Generation:** Computes aggregate scores per page and extracts dynamic context snippets with highlighted `<mark>` tags around matched query keywords.
- **REST Discovery Endpoint:** Accessible via `GET /api/crawler/search?q={query}`, enabling sub-millisecond full-text queries over indexed web content.

12. Information Retrieval: Okapi BM25 Ranking & Linguistic Stemming

Linguistic Stemming (Porter Stemmer)

Simple exact-word matching fails when searching web corpora due to inflected variants (e.g. searching "computing" misses documents containing "computed", "computes", or "computer"). The crawler implements a cascading morphological stemmer:

- **Rule-Based Reduction:** Strips plural, past-participle, and agent suffixes (ing, ed, er, or, s, ies, ation, ment, al).
- **Stem Inverted Index:** Both indexed document terms and search query tokens are mapped to their morphological stems, enabling recall of all linguistic derivations.

Okapi BM25 Information Retrieval Formula

Rather than naive term frequency, textual relevance is computed using the industry-standard **Okapi BM25** probabilistic relevance framework:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Where:

- $f(t, D)$ is the term frequency of term t in document D .
- $|D|$ is the length of document D in words, and avgdl is the average document length across the crawled corpus.
- $k_1 = 1.2$ calibrates non-linear term saturation (preventing keyword-stuffed documents from dominating results).
- $b = 0.75$ controls document length normalization penalty.
- $\text{IDF}(t) = \ln\left(1 + \frac{N - n(t) + 0.5}{n(t) + 0.5}\right)$ quantifies the informational rarity of term t across N crawled documents.

13. Graph Link Analysis: Google PageRank Authority Algorithm

A high textual relevance score does not necessarily guarantee page quality or credibility. The crawler computes **Google PageRank** link authority to evaluate global node importance based on the web graph's hyperlink topology:

Mathematical Model

Modeled as a random surfer Markov chain:

$$\text{PR}(p_i) = \frac{1 - d}{N} + d \cdot \sum_{p_j \in M(p_i)} \frac{\text{PR}(p_j)}{L(p_j)}$$

Where:

- $M(p_i)$ is the set of pages linking to page p_i (in-degree / backlinks).
- $L(p_j)$ is the number of outbound hyperlinks from page p_j .
- $d = 0.85$ is the damping factor representing the probability that a surfer follows links rather than teleporting to a random page.
- N is the total count of crawled pages.

Numerical Power Iteration

- **Dangling Node Redistribution:** Pages with zero outbound links ($L(p_j) = 0$) leak probability mass. The engine redistributes dangling node mass equally across all nodes on each iteration step: $\text{Dangling Mass} = \sum_{p \in \text{Dangling}} \text{PR}(p)$
- **Convergence:** Iterates until the Euclidean difference between successive probability vectors falls below $\epsilon = 10^{-4}$ (typically 15–25 iterations).
- **Normalized Authority Scaling:** Raw stationary distribution values are scaled non-linearly to an intuitive 0 - 100 authority index: $\text{Score}_{\text{PageRank}} = \left(\frac{\text{PR}(p_i)}{\max(\text{PR})}\right)^{0.6} \times 100$

Multi-Strategy Hybrid Ranking (Composite)

When queries are executed, the engine supports multiple ranking modes:

1. **COMPOSITE (Default)**: Harmonizes textual relevance with topological link authority: $\text{Score}_{\text{Composite}} = 0.65 \times \text{BM25}_{\text{norm}} + 0.35 \times \text{PageRank}$
2. **PAGERANK**: Orders results purely by graph authority and backlink centrality.
3. **BM25**: Orders results purely by probabilistic textual relevance.
4. **WORDS**: Orders results by exhaustive document depth / word count.
5. **NEWEST**: Orders results by crawl timestamp.

14. "Fetch Everything" Aggregated Corpus Analytics & Taxonomy

When a search query or topic keyword is submitted, the engine provides an exhaustive, multi-dimensional discovery summary across the entire indexed corpus:

- **Total Corpus Occurrences**: Counts every single occurrence of the query stem across all indexed documents, titles, headings, and meta tags.
- **Dynamic Category Distribution**: Automatically groups matching documents by high-level semantic taxonomy (Quantum Computing, Artificial Intelligence, Cloud & Infrastructure, Sports & Athletics, Science & Research, World News).
- **Related Topic Entities & Co-occurrences**: Analyzes high-frequency terms co-occurring in the matched documents, suggesting related exploration tags (e.g. searching "quantum" highlights "qubits", "superposition", "algorithms").
- **Prefix Autocomplete**: Maintains an in-memory vocabulary trie/index that delivers matching keyword suggestions and corpus document frequencies in $< 1 \text{ ms}$ as users type (GET /api/crawler/autocomplete?prefix=...).

15. Summary of Architectural Verification & Deliverables

Requirement	Implementation Artifact	Verification Result
URL Frontier Queue	`UrlFrontierQueue.java` (Redis List `crawl:queue`)	Verified FIFO traversal, thread-safe task dispatching.
Visited Set Deduplication	`VisitedUrlService.java` (Redis Set `crawl:visited`)	0 duplicate URL fetches across cyclic and self-referencing loops.
Content Deduplication	`ContentDeduplicationService.java` (SHA-256)	Cloned articles detected and redundant subtree exploration suppressed.
Robots.txt Engine (RFC 9309)	`RobotsTxtService.java`	Longest-match prefix precedence obeyed; ` <meta >`="" honored.<="" name="robots" td=""/>
XML Sitemap Ingestion	`SitemapService.java`	Standard `/sitemap.xml` and robots.txt declarations auto-discovered.
PageRank Link Authority	`PageRankService.java`	Power iteration ($d=0.85$,

Requirement	Implementation Artifact	Verification Result
		$\epsilon=10^{-4}$) converged; scores displayed in graph.
Information Retrieval	<code>`ContentSearchService.java`</code> (BM25 + Stemmer)	Multi-field weighted scoring, stem expansion, context snippet generation.
Vocabulary Autocomplete	<code>`CrawlerController.java`</code> (<code>/autocomplete`</code>)	Sub-millisecond prefix suggestion with frequency counts.
Data Export	<code>`GET /api/crawler/export?format=json`</code> or <code>`csv`</code>	Full page metadata, headings, keywords exported cleanly in JSON and CSV.

API / Command Reference & Test Results

REST Endpoints Summary:

- `POST /api/crawler/start`: Asynchronously starts crawl session from seed URLs
- `POST /api/crawler/stop`: Halts running crawler worker pool
- `POST /api/crawler/reset`: Flushes Redis queue, visited set, and page cache
- `POST /api/crawler/crawl-single?url=...`: Synchronously crawls 1 URL (with Redis caching)
- `GET /api/crawler/status`: Retrieves real-time crawling metrics and throughput
- `GET /api/crawler/pages`: Returns collection of all crawled pages and metadata
- `GET /api/crawler/visited`: Returns complete set of visited URLs from Redis
- `GET /api/crawler/queue`: Inspects URLs waiting in the frontier queue

JUnit 5 Test Suite Results:

- `UrlValidatorTest.testValidHttpUrls`: PASSED
- `UrlValidatorTest.testTrailingSlashRemoval`: PASSED
- `UrlValidatorTest.testDisallowedSchemes`: PASSED
- `UrlValidatorTest.testDisallowedExtensions`: PASSED
- `UrlValidatorTest.testDomainBoundaryRestriction`: PASSED
- `UrlValidatorTest.testRelativeUrlResolution`: PASSED
- `CrawlerServiceTest.testFrontierQueueOperations`: PASSED
- `CrawlerServiceTest.testVisitedSetDuplicatePrevention`: PASSED

Learning Outcomes

- Engineered an autonomous web crawler in Spring Boot implementing queue-based Breadth-First Search (BFS).
- Utilized Redis Lists (`RPUSH/LPOP`) to establish a distributed URL frontier queue decoupled from worker threads.
- Leveraged Redis Sets (`SADD`) to achieve race-free $O(1)$ atomic visited-set deduplication across concurrent workers.
- Integrated Jsoup for robust HTML DOM parsing, relative URL canonicalization, and metadata extraction.

- Observed a 29x latency speedup by caching parsed document metadata in Redis.
- Conducted JMeter load testing evaluating concurrency scaling, throughput limits, and response latencies.